

NP-Complete

Is this problem hard?

Easy & Hard Problem

- What is **difficulty** of problem?
 - Hard for **you** to design an algorithm for the problem?
 - Hard for **a computer** to solve (run the designed algorithm) the problem?
 - Difficult in a sense that it is going to take a lot of resources, (time, space)
- We will consider the question **How hard this problem is for a computer.**

Basic Intuition

- Easy problem → a computer can solve using little time, i.e., anything we can solve within $O(T(n))$ where $T(n)$ is polynomial
- Hard problem → Anything else... (maybe)
- More questions
 - Is there something unsolvable?
 - How to know whether the problem is hard?

Goal

- Understand several classes of problem, each with different difficulty.
- Can use a tool to decide which problem is harder than another problem.

Unsolvable Problem

Hardest Problem, guaranteed failing.

Unsolvable

- Formally, it's called an **undecidable problem**
- A problem that we can prove that there is no algorithm for the problem.
- Example:
 - Halting Problem.

Halting Problem

- Input: Given a program P and its input S
- Output:
 - “Yes” when P finishes
 - “No” when P loops forever
- To have an algorithm for *halting problem*
 - The algorithm must be able to correctly answer this question for every possible P and S .

Proving that Halting is undecidable

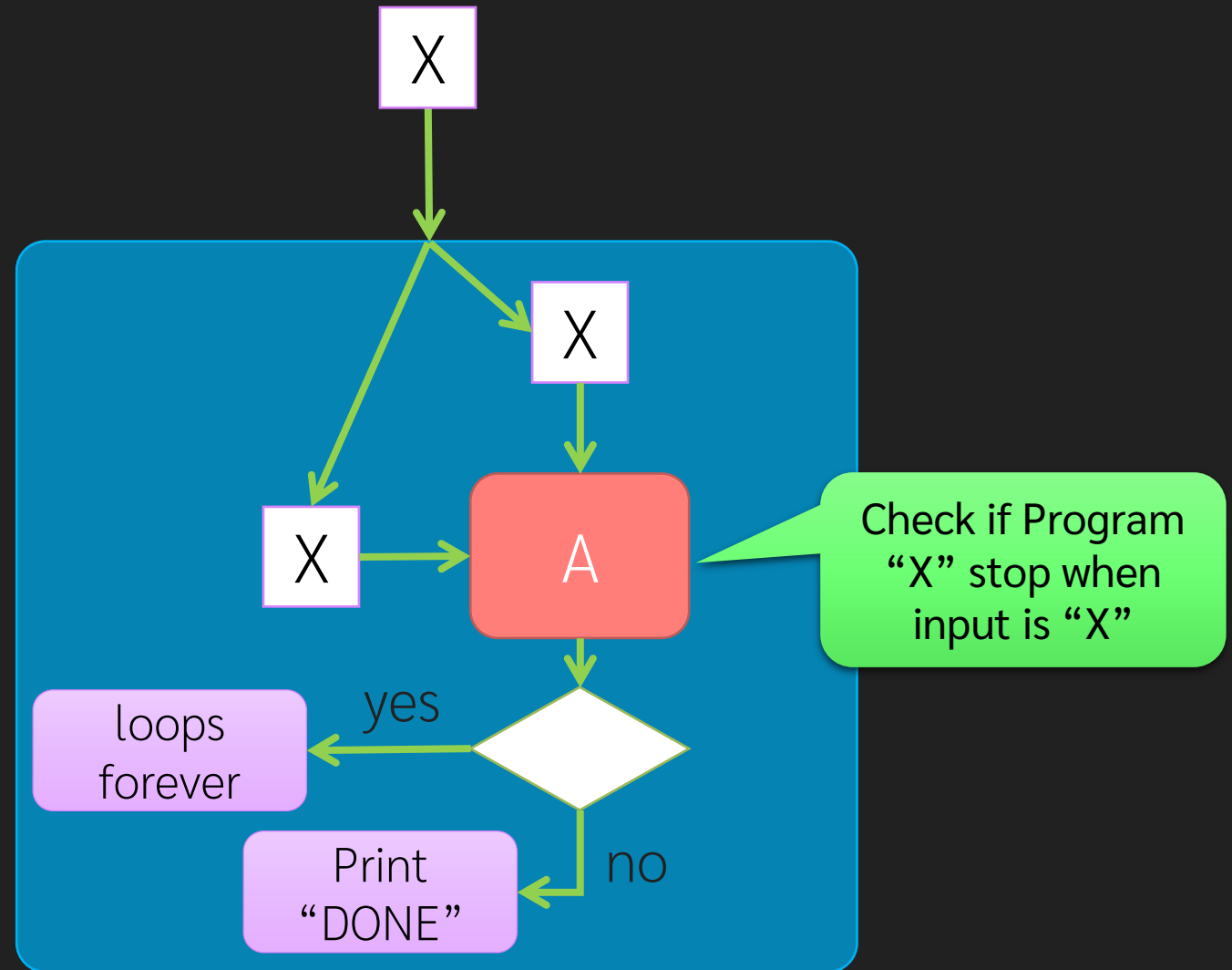
- Proof by contradiction
- Let us assume that there is an algorithm for the problem. Let's say `bool A(P, S)` is a function that solve this problem.
 - Return true when a program `P`, will not run forever when it takes `S` as in input
- We construct another function `kaboom(x)` which takes only one input
 - Assume that kb.cpp has this function

```
bool kaboom(x) {  
    if (A(x,x)) {  
        while (true) do ;  
    } else {  
        printf("DONE!");  
    }  
}
```

Kb.cpp

Kaboom(X)

- Check if X(X) stop
 - If yes, Kaboom will loops forever
 - If no, Kaboom will stop
- What is the result of Kaboom when we feed the file kb.cpp as its input
 - The answer cannot be **yes**
 - It can't be **no** as well



Complexity Class

Class of complexity

- We will introduce classifications of problems, by defining a set of problems such that problem in the same set is of similar hardness.
- Ideally, we would prefer to have nice partition of problem.



- In reality, this is not the case
 - There are several sets, each are defined by hardness, but the boundary is not definite.
 - Each is called a **complexity class**



Some class

- These are some well-known classes

Class	Intuition
P	Easy problem
NP	Most problems we really cared
NP-Hard	Very hard problems
NP-Complete	Special class of problem, very important. We believe that it is hard, but we don't really know
Undecidable	Super hard, impossible problem. Proven that there is no correct algorithm.

Decision Problem

- Decision problem = a problem whose output is either YES or NO
- Ex,
 - Given a graph G , does it have Euler circuit?
 - Given a list of integers, is their GCD equal to 1?
 - Given coordinate of two 2D boxes, do they intersect?
 - Given a number X , is it a prime?
- For simplicity, we will consider only decision problems
 - Classification of problem is done on decision problems only.

Function problem

- Any problem whose output is not just YES or NO.
 - Formally, the problem is defined as finding a function that maps a string of input into a string of output.
- Ex.
 - Given an array, what is the largest element in the array?
 - Output is a value (not just YES or NO)
 - Given a graph G and two vertices u and v , how many path are there from u to v ?
- Most of problem we have seen so far in this class are function problem; the output can be something other than YES or NO.

Equivalence to Decision Problem

- All function problems can be solved by decision problems
 - For polynomial output, using polynomial time
- Example, consider a shortest path problem
 - Input are a graph G , a start vertex, and a target vertex.
 - Function problem: Output is the summation of weight of the shortest path.
 - Decision problem: we will use several instance of decision problems.
 - Ask whether there exist a path from s to u in G such that the weight is equal to X
 - Start by asking for $X = 1$, then 2, 3, 4, ...
 - Or better, use binary search.
 - Eventually, we know the minimal X and hence the minimal path

Decision problem is everything!

Why decidable problem?

- Easier to prove, there is only two cases of output.
- Polynomially equivalent to function problem.

P and NP

Hard vs Easy

- We do not really have an absolute, direct way to indicate that a single problem is hard or easy.
- But we have a method to compare difficulty of two problems, which one is harder, which one is easier, or both is of the same hardness.
- This revolves around the concept of reducibility.

Reduction

- **Reduction**: solve problem **A** by transforming it into another problem **B**.
- The problem **A** is said to be reducible to the problem **B** when every possible instance of **A** can be **easily transformed** into instances of **B** and the solution of the original instance of **A** can be **calculated easily** from the solution of those instances of **B**.
- “**Easily**” in the above sentence means it can be done in **polynomial time**.

Example: Reduce K^{th} smallest into sorting.

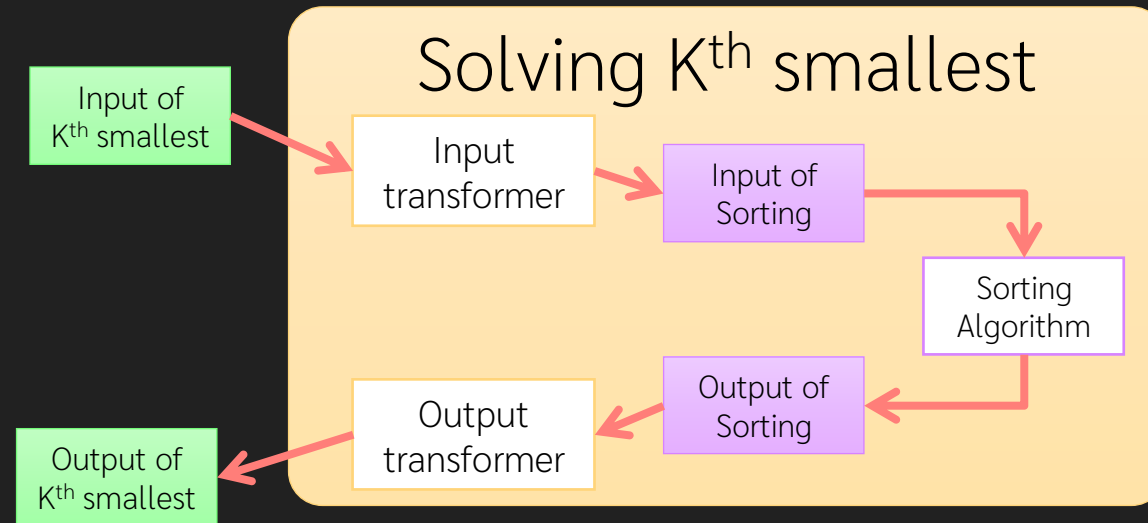
K^{th} smallest Problem

- Input: $D[1..n]$ and K
- Output: x such that $D[x]$ is the k^{th} smallest value in D

Sorting Problem

- Input: $S[1..n]$
- Output: Sorted value of S , i.e., $S[i]$ is the i^{th} smallest value in S .

Reduce K^{th} smallest to Sorting by send D as an input S of the sorting problem. Let S be the output of the sorting problem, we answer the K^{th} smallest problem by using $S[k]$.



Polynomially Reducible

- Let X be an instance of problem A
- Let $f(X)$ be a function that transform an instance X of the problem A into (maybe several) instances of the problem B .
- If $f(X)$ uses polynomial time to the size of X , we say that A is polynomially reducible to B
- Polynomially Reducible is what we prefer because we can chain polynomial function and still get a polynomial function.

Comparing “hardness” of problem

- Given an instance of problem A
- If that instance of A can be transformed into an instance of problem B
- Which one is harder?
 - A or B?

Reducible is ordering of hardness

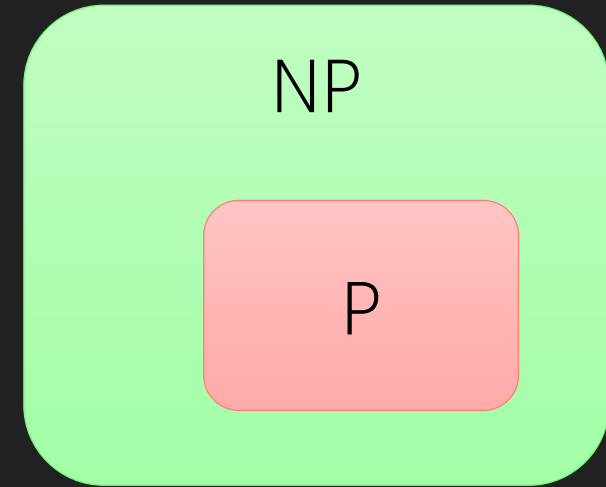
- When a problem A can be reduced to a problem B, we say that B is **not easier** than A.
 - Because, essentially, A can be considered as a **subset**, or something **embedded** in B.
 - Hence, algorithm for B is **larger**, more **capable**. So, B is harder (not easier)
- Thinking of a task of **driving any kind of vehicle** vs **driving a car**. A person that perform this task is an algorithm.
 - If the problem is to **drive a car**, we can definitely solve it by someone that **can drive anything**.
 - If the problem is to **fly a helicopter**, we might not be able to solve it by someone that can **drive a car**.

P and NP

- Class of problems
- P
 - set of problems that we have polynomial time algorithm or better
- NP
 - Informal: “**exponential time**”; set of problems that we have exponential time algorithm or better
 - More formal: “**polynomially verifiable**”; set of problems that we can verify, in polynomial time, whether, when the answer is “YES”, it is correct (given right evidence)
 - Actual: “**non-deterministically provable**”; set of problems that can be solved by non-deterministic Turing machine.
 - i.e., every problem we have seen so far.

Example

- P
 - MSS, Sorting, Shortest Path, MST
- NP
 - MSS, Sorting, Shortest Path, MST is also NP
 - TSP, 01-knapsack, graph coloring, longest path
 - And much more



More Example

- Shortest Path
 - Given a weighted graph G and two vertices u, v , find shortest path from u to v
 - Input: a graph G and two vertices u and v
 - Output: a path
- Is in P
 - Dijkstra's algorithm runs in P . (recall that it is $O(E \log V)$)
- Is in NP
 - given a graph and a path, we can verify that the path is the shortest path
 - By running Dijkstra ourselves and compare the length with the given path

More Example

- Graph isomorphism
 - Given two graphs, decides whether they are isomorphic.
 - Input: two graphs
 - Output: YES or NO
- Is in NP
 - Can be verify if a mapping is given
- It is not known to be in P
 - We have not found a polynomial time algorithm for it
 - HAVE NOT but not CANNOT

Hard vs Easy again

- P is the set of easy problems
 - Easy because we can solve any of them in reasonable time, because we have algorithm for any of them that runs in polynomial time.
 - Theoretically, a problem with $O(n^{9999})$ algorithm is in P even it takes very long time to compute.
 - In practice, we rarely see that kind of polynomial.
- NP has easy problems + something else
 - Those something else are believed to be hard problems

Is NP a set of hard problem?

- No, something in NP is easy
- No, something hard is not in NP
 - Try graph non-isomorphism
 - The verifiable definition for NP is when the answer is “YES”

NP-Hard

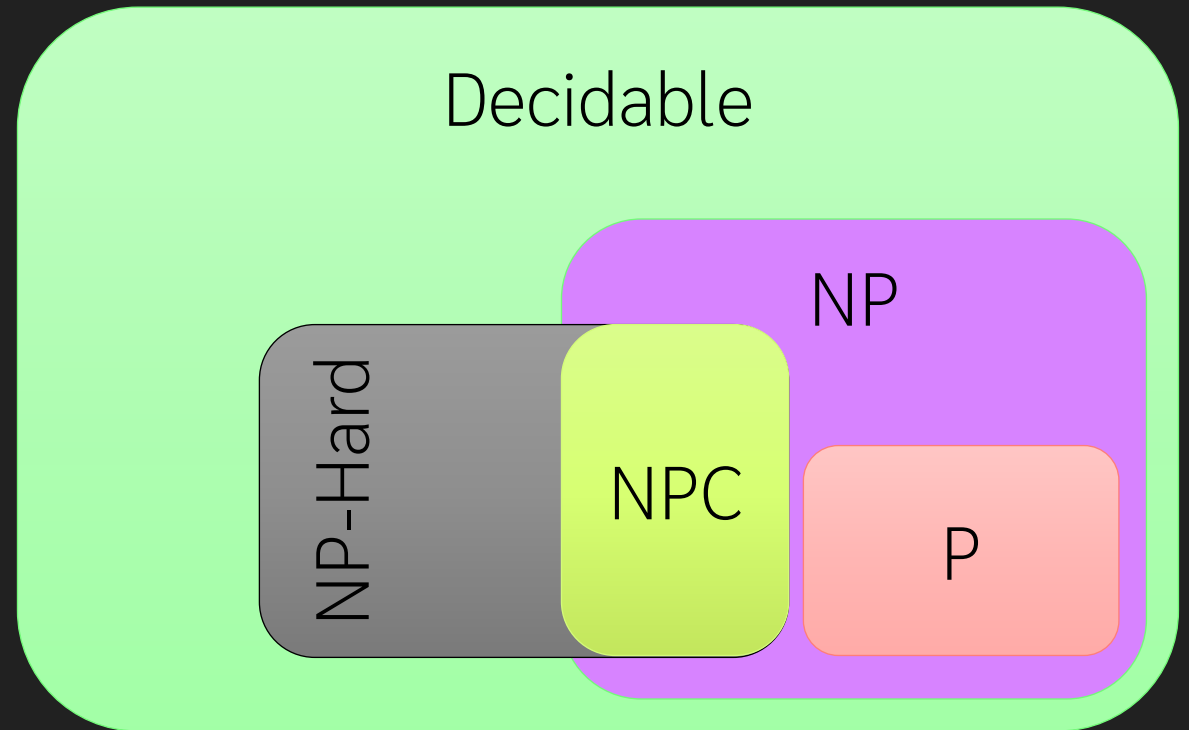
- Something that is really hard, comparing to NP.
- Definition, NP-Hard is a problem that any problem in NP can be reduced to.
- Cook-Levin Theorem states that
 - Boolean satisfiability problem is NP-Hard
 - Boolean satisfiability problem is in NP.

Boolean Satisfiability Problem

- Given a Boolean expression in CNF (sum of product)
 - $(x \vee y \vee \bar{z}) \wedge (\bar{x} \vee y \vee w) \wedge (\bar{y} \vee \bar{w})$
- Can we assign a truth value to each variable such that the expression is evaluated as TRUE.
 - For above, the answer is YES by assigning y as TRUE and w as FALSE
- Intuitively, any digital circuit can be written as a Boolean expression and NP is something computable by a Turing machine which is, loosely speaking, a model of a digital circuit. So, anything computable by a Turing machine is Boolean expression and finding whether it is YES is the same as the Boolean Satisfiability Problem.
 - This is very coarse, so much hand-waving and not really correct proof.

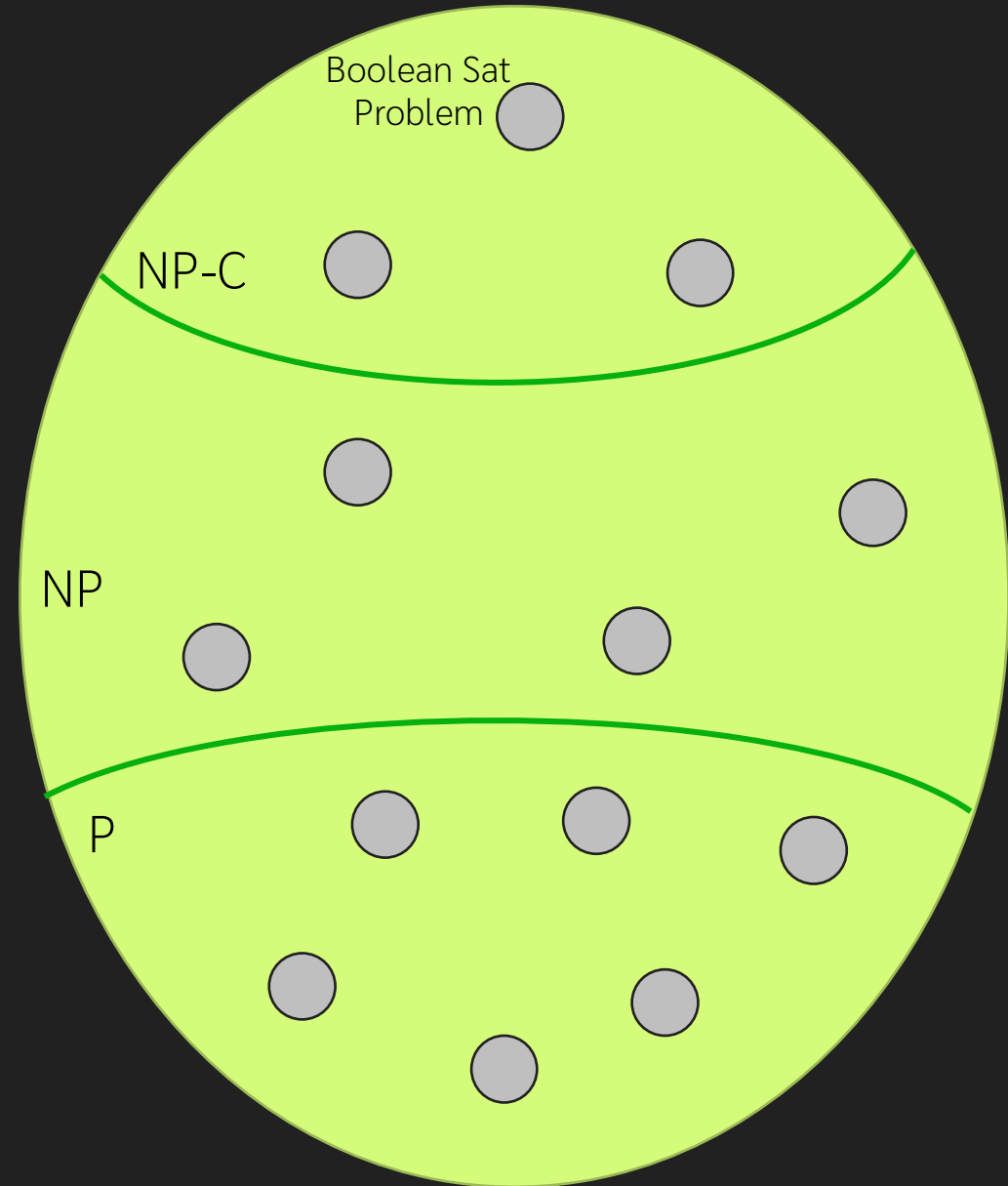
Finally, the NP-Complete

- NP-Complete
 - An NP-Hard that is in NP
- It has so much profound effect in complexity theory



NP-Complete

- Since NP-C is a subset of NP-Hard, every problem in P can be reduced into an NP-C problem.
- Since NP-C is in NP, every problem in NP-C can also be reduced into an NP-C problem.



Current Status

- Several problems are known to be in P
 - To prove that a **problem is in P**, we just need to **find an algorithm** that solve the problem in polynomial time.
 - To prove that it is **NOT in P**, we must show that **polynomial algorithm is not possible**. This is very hard to do.
- Several problem is known to be in NP
 - To prove that a **problem is in NP**, we just need to **find an algorithm** that check if the solution of YES is correct.
 - To prove that it is NOT in NP, we must show that **polynomial verification algorithm is not possible**.
- Several problem is known to be in NP-Hard
 - This one is harder to prove. We need to show that every problem in NP can be reduced to it.

The ultimate question $P = NP$?

- Currently, we **HAVE NOT** found any polynomial algorithm for any problem in **NP-C**
 - If we can find just one algo for just one problem, every problem in **NP-C** and also every problem in **NP** can be solved in polynomial time.
 - In other words, we have proven that $P = NP$, everything in **NP** is easy.
- We also **HAVE NOT** proven that such algorithm cannot exist.
 - If we can prove that just one problem in **NP-C** cannot have polynomial time algorithm, every problem in **NP-C** must also have no polynomial time algorithm as well.
 - In other words, we will know that $P \neq NP$, everything in **NP-C** is definitely hard.